

The NASM Preprocessor

- **Power of the Preprocessor:** NASM includes a very powerful preprocessor similar to C, but more flexible.
- **Single-line Macros: %define**
 - `%define CTRL_L 12`
 - Used for constants and simple aliases.
- **Multi-line Macros: %macro and %endmacro**
 - Allows for "pseudo-instructions."
 - Example: A macro to print a string can wrap multiple `mov` and `syscall` instructions.

Anatomy of a Multi-line Macro

- **Syntax:** `%macro Name Number_of_Args`
- **Argument Access:** Use `%1`, `%2`, etc.
- **Local Labels:** Labels inside macros must start with `%%` to avoid name collisions when the macro is used multiple times.
 - `%%loop_start`:
- **Overloading:** You can define multiple macros with the same name but different argument counts.

Conditional Assembly

- **Directives: %if, %elif, %else, %endif.**
- **Use Cases:**
 - Writing code that supports multiple Operating Systems (e.g., %ifdef LINUX).
 - Debugging blocks: %ifdef DEBUG ... %endif.
- **%assign vs. %define:**
 - %assign handles numeric expressions and can be re-assigned (useful for loop counters in macros).

Interfacing with C

The "law" for how functions communicate in Linux.

Register Hierarchy:

- Arguments: rdi, rsi, rdx, rcx, r8, r9 (in order).
- Return Value: rax.
- Caller-Saved: rax, rcx, rdx, rsi, rdi, r8-r11 (assume they get destroyed).
- Callee-Saved: rbx, rbp, r12-r15 (you MUST restore these if you use them).
- External Symbols: extern printf — tells NASM the symbol is defined elsewhere.

Managing the Stack Frame

■ The Prologue:

- `push rbp` ; Save old base pointer
- `mov rbp, rsp` ; Set new base pointer
- `sub rsp, 16` ; Allocate 16 bytes for locals

■ The Epilogue:

- `mov rsp, rbp` ; Deallocate locals
- `pop rbp` ; Restore old base pointer
- `ret`

- **Alignment: The stack must be 16-byte aligned before a call instruction. Failure results in a Segfault during SIMD operations in C libraries.**

Floating Point: SSE and AVX

- **Modern SIMD (SSE/AVX):**
- **Registers:** xmm0 through xmm15 (128-bit) or ymm0 (256-bit).
- **Scalar Ops:** addss (Scalar Single-precision), addsd (Scalar Double-precision).
- **Packed Ops:** addps (Packed Single-precision) — performs 4 additions at once.
- **Instruction Suffixes:** ss = 32-bit float, sd = 64-bit double.